PRODUCT REQUIREMENTS DOCUMENT

Outdoor Vision CV App

GUI Home Page - Product Requirements Document

VERSION	DATE	STATUS
1.0	August 23, 2026	Approved for Implementation
OWNER	**APPROVER**	**PLATFORM**
Dave Vercher	Dave Vercher	Windows Desktop

DOCUMENT INTENT

Implementation-ready requirements for the Outdoor Vision CV App home page: first-run library setup, persistent project management, responsive project cards, statistics refresh, safe filesystem operations, and project-library relocation.

This document is the authoritative requirements baseline for the next GUI update.

Contents

Approved scope APPROVED

Build the Windows desktop home page, first-run project library setup, persistent project management, responsive cards, project statistics refresh, and minimal library-location Settings.

Scope boundary OUT OF SCOPE

Do not implement the project workspace, image sorting integration, annotation tools, dataset export, or model training in this update.

1. Document Purpose

This document defines the requirements for the next update to the existing JPEG Dataset Sorter desktop GUI. The update will introduce a new application home page and persistent project management. It is intentionally limited to the home page, first-run project-library setup, project creation and management, and the minimum Settings functionality needed to relocate the project library.

This document is intended to be sufficient implementation context in a future coding session. The implementer should inspect the existing repository before making changes, preserve the current JPEG sorting functionality, and place that functionality behind the project-level experience in a later update. The existing sorter does not need to be integrated into a project workspace as part of this update.

2. Product Vision and Scope

The longer-term product is a Windows desktop application that guides an untrained user through an end-to-end computer-vision dataset-preparation workflow. The current JPEG folder sorter will eventually become one module inside a project, alongside future tools for filename standardization, dataset selection, bounding-box annotation, YOLO annotation generation, dataset packaging, and local model training.

This update implements only the entry point into that future workflow. The home page must allow the user to create projects, view and search previously created projects, open a project, rename or delete a project, refresh project statistics, and access the setting used to relocate the master project library.

2.1 In Scope

- A Windows desktop application home page.
- First-run selection of a parent location for the project library.
- Automatic creation of a master project folder named `outdoor_vision_CV`.
- Persistent app-level knowledge of the master project folder and registered projects.
- Creation of empty project folders.
- Full-width project cards with project statistics and actions.
- Project search, sorting, responsive pagination, and manual statistics refresh.
- Opening a project from its card, with only the transition boundary defined in this update.
- Renaming and permanently deleting projects through the GUI.
- Detection and handling of missing project folders.
- A minimal Settings view for moving the complete project library.
- A private statistics interface that later project tools can update.

2.2 Out of Scope

- Designing or implementing the screen displayed after a project is opened.
- Integrating the current JPEG sorter into a project workspace.
- Image import, class-folder creation, filename standardization, dataset selection, annotation, YOLO export, or training.
- Finalizing the internal folder structure of a project.
- Automatically calculating labeling progress from files.
- Supporting operating systems other than Windows.
- Importing arbitrary folders as projects.

- Recognizing subfolders not created and registered by this application as projects.
- Merging project libraries.

3. Terminology

- Parent location: A user-selected directory in which the application creates the master project folder.
- Master project folder / project library: The application-managed, user-visible folder named `outdoor_vision_CV`.
- Project: A direct child folder of `outdoor_vision_CV` that was created and registered through the application.
- Project registry: Private app-level data that identifies projects created by the application and stores their display statistics. It must not be stored in the user-visible project folders.
- Labeled image: In the future labeling workflow, an image that either has at least one bounding box or is explicitly tagged `Empty`.

4. Platform and Design Direction

4.1 Platform

- The application must run as a local Windows desktop app.
- Folder-selection controls must use the standard Windows folder picker.
- Clicking a project folder path must open that folder in Windows File Explorer.
- All project creation, renaming, deletion, and relocation actions must perform real Windows filesystem operations. The GUI must not present a renamed, deleted, or moved state unless the corresponding filesystem operation succeeds.

4.2 Visual Direction

- The home page must use a compact, minimal, Obsidian-inspired dark theme.
- Primary surfaces should use blacks, charcoal, and dark gray tones, with restrained borders and contrast.
- The interface should feel lightweight and less visually dense than Label Studio.
- Text, controls, focus indicators, progress bars, disabled states, and error states must remain legible and accessible against the dark background.
- The layout must resize cleanly when the user changes the application-window dimensions.

5. Persistence and Filesystem Model

5.1 App-Level Configuration

The application must persist the following information in a private, app-appropriate local configuration location:

- The absolute path to `outdoor_vision_CV`.
- The registry of projects created through the GUI.
- For each project: its name, absolute folder path, last-edited timestamp, total-image count, labeled-image count, and any identifiers needed for reliable internal references.
- Any additional home-page preferences required by the implementation.

This private application data must not be placed in the user-visible master project folder or individual project folders. The implementation should use a normal Windows per-user application-data location rather than relying on a configuration file beside an installed executable, because installed application directories may not be writable.

The application must write registry/configuration updates atomically where practical so that an interrupted write does not corrupt the full registry. A corrupt or unreadable registry must produce a clear recovery message rather than crashing silently.

5.2 User-Visible Files

- `outdoor_vision_CV` and all project folders are ordinary, user-accessible Windows folders.
- Every registered project folder must be a direct child of `outdoor_vision_CV`.
- All future user-facing project files and folders must remain underneath the corresponding project folder.
- A project created in this update must initially be an empty folder.
- Project folders must not contain private GUI metadata solely for the purpose of powering the home page.

5.3 Project Recognition

- The application must list only projects found in its private project registry.
- It must not automatically treat every subfolder of `outdoor_vision_CV` as a project.
- A folder manually added to the master folder must not appear on the home page.
- A registered project whose folder is missing must remain visible in a disabled missing-folder state until the user removes its stale registry entry.

6. First-Run Experience

6.1 Trigger

The first-run setup must appear when the application has no valid saved master-folder location. Normal home-page functionality must not be available until setup succeeds.

6.2 Flow

1. Explain briefly that all Outdoor Vision CV projects will be stored together in a user-accessible project library.
2. Ask the user to choose a parent location using the Windows folder picker.
3. Show the resulting path before confirmation: `<selected parent>\outdoor_vision_CV`.
4. After confirmation, create the `outdoor_vision_CV` folder.
5. Save its absolute path in private app configuration.
6. Open the home page.

6.3 Validation and Failure Handling

- The selected parent location must exist and be writable.
- If `<selected parent>\outdoor_vision_CV` already exists, setup must not merge with it or overwrite it. The user must be asked to select another parent location.
- If folder creation or configuration persistence fails, show a specific error and keep the user in setup without recording a partially completed configuration.
- Cancelling the folder picker must return the user to the setup prompt without creating files.

6.4 Subsequent Launches

- On subsequent launches, the app must load the saved master-folder path automatically and go directly to the home page when the location remains valid.
- If the master folder is missing or inaccessible, the app must show a blocking recovery state that explains the problem and allows the user to select a new valid location. It must not silently create a replacement library or discard the existing registry.

7. Home Page Layout

The home page must contain:

- A recognizable application title/header.
- A prominent `New Project` action.
- A project-name search field near the top of the project list.
- A sort control supporting `Last edited` and `Project name`.
- A visible `Refresh` action for project statistics.
- A `Settings` action.
- The paginated project-card list.
- Pagination controls at the bottom of the list.

The project tools/modules must not appear on the home page. They will be shown only after a project is opened in a future update.

7.1 Empty Library State

When no projects exist, the page must show a concise empty state that explains that no projects have been created and directs the user to `New Project`. Search, sorting, and pagination must not display misleading active results in this state.

8. New Project

8.1 User Input

Selecting `New Project` must open a focused dialog that requests:

- Project name.
- Project folder location, represented by the fixed master-folder location. The user may be shown the destination and may use a Browse-style control to view/select the configured library context, but all projects must remain direct children of `outdoor_vision_CV`. The user must not create a project outside the configured master folder.

Because the master location is globally configured, the creation dialog should clearly show the final folder path and avoid implying that each project can use an unrelated destination.

8.2 Project-Name Rules

- Length must be 1-25 characters.
- The name must be a valid Windows folder name.
- Characters invalid in Windows folder names must be rejected, including <, >, :, ", /, \, |, ?, and *.
- Names must not end with a space or period.
- Windows-reserved device names must be rejected, including reserved names with extensions where Windows would still treat the base as reserved.
- Project names must be unique within the library using Windows-style case-insensitive comparison. For example, `Hog Detector` and `hog detector` are duplicates.
- Creation must also fail if any filesystem entry with the proposed name already exists under `outdoor_vision_CV`, even if that entry is not registered as a project.
- Validation feedback must be shown before creation when possible and must tell the user how to correct the name.

8.3 Creation Behavior

On confirmation, the application must:

1. Revalidate the name and destination to protect against changes made while the dialog was open.
2. Create `<master folder>\<project name>` as an empty folder.
3. Add the project to the private registry only after folder creation succeeds.
4. Initialize statistics to `0 images` and `0 / 0 labeled`.
5. Initialize `last edited` to the successful creation time.
6. Close the dialog and show the new project at the appropriate position under the active sorting mode.

If registry persistence fails after folder creation, the app must clearly report the failure and avoid leaving an untracked project folder where practical. Any rollback failure must be disclosed to the user.

9. Project List and Cards

9.1 List Organization

- Projects must be presented as full-width row cards.
- The default sort must be most recently edited first.
- The user must be able to sort projects alphabetically by project name.
- Alphabetical sorting must use case-insensitive user-friendly comparison.
- The sort control must clearly communicate its active mode. If direction is configurable, both ascending and descending states must be explicit; otherwise alphabetical means A-Z and last edited means newest first.

9.2 Search

- The search field must filter by project name using case-insensitive substring matching.
- Filtering should update as the user types without requiring a separate submit action.
- Clearing the search restores the full project list.
- When no project matches, show a specific no-results state rather than the empty-library state.

- Search is applied before pagination, and changing the search must return the list to its first page.

9.3 Card Contents

Each normal project card must display:

- Project name as the prominent heading.
- Full absolute project-folder path.
- Last-edited date/time.
- Total image count.
- Labeling progress bar.
- Labeled count, total count, and percentage, e.g. `325 / 500 labeled - 65%`.
- An ellipsis menu aligned to the right.

For a new or empty project, display `0 images`, `0 / 0 labeled`, and a visually empty progress bar. Do not display an undefined or divide-by-zero percentage; the recommended percentage display is `0%`.

9.4 Card Interaction

- Clicking a normal card must open that project immediately.
- This update only needs to provide a clean project-open navigation boundary; the contents of the project screen are out of scope.
- Clicking an interactive child control such as the path link or ellipsis must not also trigger card opening.
- Clicking the displayed folder path must open the physical folder in Windows File Explorer.
- If File Explorer cannot open the folder, show an error and keep the application usable.

10. Responsive Pagination

- Pagination controls must include `Previous`, `Page X of Y`, and `Next`.
- The number of cards per page must be calculated from the vertical space available in the resized application window.
- The minimum supported window size must be designed to show at least five complete project cards whenever five or more matching projects exist, plus the required header/filter and pagination controls.
- Resizing the window must recalculate page capacity without clipping cards or controls.
- When capacity changes, keep the current first visible project on screen where practical; otherwise clamp to a valid page.
- `Previous` and `Next` must be disabled at the first and last page respectively.
- A single-page result set may show disabled pagination controls for layout consistency.
- Search, sorting, project creation, project deletion, and project renaming must update page counts and clamp or reset the current page appropriately.

11. Labeling Progress and Image Statistics

11.1 Definitions

- `total images` is the number of files with a `.jpg` or `.jpeg` extension found recursively anywhere beneath the project folder. Matching must be case-insensitive.
- `labeled images` is a value maintained by future project tools. An image will eventually count as labeled when it has at least one bounding box or is explicitly tagged `Empty`.
- `labeling percentage` is `(labeled images / total images) × 100`, displayed as a whole-number percentage unless a later design requires more precision.
- When total images is zero, display zero progress.
- The UI must guard against inconsistent stored values. A labeled count below zero is treated as zero; a labeled count above the current total must not render progress beyond 100%. The implementation should preserve/report the inconsistency for later reconciliation rather than silently damaging valid future data.

11.2 Automatic Updates from Future Tools

The implementation must provide a documented internal service or method through which future project-level tools can update:

- Total-image count.

- Labeled-image count.
- Last-edited timestamp.

These updates must be persisted to the private registry and reflected on the home page when it is visible or next opened.

11.3 Manual Refresh

- The home page must include a visible `Refresh` button.
- Selecting Refresh must rescan every registered project folder.
- For each accessible project, recursively count `.jpg` and `.jpeg` files and update total-image count.
- For this update, Refresh must leave the stored labeled-image count unchanged because the final project/annotation structure is not yet defined.
- Refresh must also recheck whether each registered project folder exists.
- The scan must not freeze the UI. Run filesystem scanning outside the GUI event thread or process it incrementally, and show a visible in-progress state.
- Disable or debounce repeated Refresh actions while a scan is running.
- When complete, update all affected cards and show a concise completion status.
- If one project cannot be scanned, continue scanning the others and show a summary that identifies the failed project(s).
- The scanner must not follow directory links/reparse points outside the project tree in a way that could unexpectedly scan unrelated or cyclic locations.

12. Last-Edited Timestamp

The last-edited timestamp should represent the latest known meaningful change to the project, not merely the last time its card was opened.

It must update when:

- The project is created.
- The project is renamed in the GUI.
- A future GUI project tool changes project data or project files.
- Refresh detects that the project's user-visible files or folders have a newer modification time than the stored last-edited value.

Opening a project without making changes must not by itself update last edited.

During Refresh, determine the latest modification timestamp within the project tree without following unsafe/cyclic links. The project root folder timestamp may be included. If timestamps cannot be read, retain the last known value and report the scan issue.

Dates must be presented in a readable local date/time format using the user's Windows locale and time zone.

13. Ellipsis Menu

13.1 Normal Project

The ellipsis menu for an accessible project must contain:

- `Rename`
- `Delete`

13.2 Rename

- Rename must use the same validation rules as project creation.
- The application must rename the physical Windows folder, not only the displayed project name.
- Before renaming, revalidate uniqueness and check for destination collisions.
- Update the registry path/name only after the filesystem rename succeeds.
- Update last edited after a successful rename.
- If the rename fails, show a specific error, retain the original registry data, and keep the original card visible.
- A capitalization-only rename may require a safe two-step filesystem rename on Windows; it must either complete correctly or leave the original folder and registry intact.

13.3 Permanent Delete

Delete is destructive and must permanently delete the project folder and all of its contents; it is not a recycle-bin or registry-only operation for an accessible project.

The deletion flow must:

1. Open a warning dialog stating the exact absolute folder path and clearly explaining that all contents will be permanently deleted.
2. Require the user to type the exact phrase `DELETE <project folder name>`.
3. Keep the final confirmation action disabled until the phrase matches exactly.
4. Provide an obvious Cancel action.
5. After typed confirmation and final confirmation, re-resolve and validate the deletion target.
6. Confirm that the target is the registered project folder, is a direct child of the configured `outdoor_vision_CV` folder, and is not the master folder itself or any broader path.
7. Permanently delete that project folder recursively.
8. Remove the project from the registry only after successful deletion.
9. Show a short confirmation message after successful deletion.

If deletion fails or is partial, do not falsely report success. Recheck the folder, preserve the registry entry when material content remains, mark/report the error, and allow the user to retry after resolving the cause.

14. Missing Project Folders

On home-page load and manual Refresh, the application must verify every registered project path.

If a project folder has been moved, renamed, or deleted outside the app:

- Keep its card visible.
- Gray out the card and display `Project Folder Missing` prominently.
- Disable clicking the card to open the project.
- Disable the folder-path link.
- Do not silently recreate the folder.
- Keep the ellipsis menu available with a `Remove from list` action.
- `Remove from list` must show a confirmation explaining that only the stale registry entry will be removed because no folder was found.
- Removing the stale entry must not attempt to delete any filesystem location.

Rename and permanent Delete actions must not be offered for a missing-folder card because the registered target cannot be safely verified.

15. Settings: Move Project Library

Settings is in scope only for changing the parent location of the master project folder. Other future settings may be absent or shown as disabled placeholders, but must not imply they are functional.

15.1 Move Flow

1. Display the current absolute path to `outdoor_vision_CV`.
2. Let the user choose a new parent location using the Windows folder picker.
3. Show the resulting destination `<new parent>\outdoor_vision_CV`.
4. Explain that the master folder and all registered project contents will be moved together.
5. Require explicit confirmation before moving.
6. Move the complete `outdoor_vision_CV` folder to the new parent.
7. Update the saved master path and every registered project path after the move succeeds.
8. Return to or refresh the home page with the new paths.

15.2 Move Validation and Safety

- The new parent must exist and be writable.

- The source and destination must resolve to different locations.
- If `<new parent>\outdoor_vision_CV` already exists, block the move. Never merge, replace, or overwrite it.
- The move must preserve the full relative contents of the master folder.
- The app must prevent project creation, renaming, deletion, opening, and Refresh while a library move is active.
- Show progress or an indeterminate busy state during the operation.
- The implementation must handle same-volume renames and cross-volume moves.
- Update registry paths only after the filesystem move completes successfully.
- If the move fails, retain or restore the original configured path wherever possible and report the exact outcome. Never claim that the library moved if files remain split between locations.
- A robust implementation should use a staged copy-and-verify approach for cross-volume moves before deleting the original. Failure recovery must prioritize avoiding data loss.

16. Loading, Error, and Accessibility Requirements

- Long-running scans and moves must not block repainting or make the application appear frozen.
- All filesystem errors must be caught and presented in plain language with the affected path and a useful next action.
- Buttons must have clear enabled, disabled, hover, and keyboard-focus states.
- Core actions and dialogs must be keyboard accessible.
- Pressing Escape should close non-destructive dialogs when safe; Enter should not bypass typed deletion confirmation.
- Text and interactive controls should meet reasonable contrast standards in the dark theme.
- The app must remain usable at its documented minimum size and common Windows display scaling levels.
- Destructive or filesystem-changing actions must not be triggered twice by rapid repeated input.

17. Home-Page State Transitions

The implementation should maintain clear states rather than inferring behavior solely from widget visibility:

- `first_run_setup`
- `library_recovery_required`
- `home_loading`
- `home_ready`
- `refreshing_statistics`
- `creating_project`
- `renaming_project`
- `confirming_deletion`
- `moving_library`
- `opening_project`
- `error`

Only actions safe for the current state should be enabled.

18. Suggested Architecture Boundaries

Exact implementation technology may remain consistent with the existing Python/Tkinter app unless the implementer has a strong repository-specific reason to refactor. Regardless of UI toolkit, separate the following concerns:

- Home-page UI/controller: Rendering, interaction, filtering, sorting, pagination, dialogs, and navigation.
- App configuration store: Master-folder path and app preferences.
- Project registry: Registered project records and saved statistics.
- Project service: Create, rename, delete, validate, and open project folders.
- Library service: First-run creation and safe library relocation.
- Statistics service: Recursive image counting, modification-time scanning, and future tool-driven updates.

Filesystem services should be testable without launching the GUI. Use `pathlib` or equivalently safe path handling, normalize paths consistently for comparison, and revalidate destructive targets immediately before acting.

19. Acceptance Criteria

The update is complete only when all of the following are demonstrably true:

1. On first launch, the user can select a parent location and the app creates `outdoor_vision_CV` there.
2. The saved library location is reused automatically after restarting the app.
3. The home page uses a responsive, Obsidian-inspired dark layout.
4. The user can create an empty project with a valid, unique 1-25-character Windows folder name.
5. Creating a project creates the real folder directly under `outdoor_vision_CV` and a persistent project card.
6. Invalid names, reserved names, case-insensitive duplicates, and existing filesystem-name collisions are rejected clearly.
7. Each project card shows name, clickable full path, last edited, total images, a progress bar, labeled/total counts, percentage, and an ellipsis menu.
8. Clicking a normal card invokes the project-open navigation boundary; clicking the path opens File Explorer without opening the project.
9. Projects can be searched by case-insensitive name substring.
10. Projects default to newest-first last-edited sorting and can be sorted A-Z by name.
11. Full-width cards paginate responsively with at least five visible at the minimum supported size.
12. Refresh recursively counts `.jpg` and `.jpeg` files without freezing the UI and leaves labeled count unchanged.
13. Future tools have a documented callable interface to update counts and last-edited values.
14. Renaming changes the physical folder and registry together, using the same name validation as creation.
15. Permanent deletion requires the exact typed phrase `DELETE <project folder name>`, validates the target, deletes only the intended project, and reports success or failure accurately.
16. A missing project folder produces a grayed-out, non-openable card labeled `Project Folder Missing`.
17. A stale missing-project entry can be removed from the registry without performing a filesystem deletion.
18. Settings can move the entire `outdoor_vision_CV` library and update all registered paths.
19. Library relocation is blocked when the destination already contains `outdoor_vision_CV`; no merge or overwrite occurs.
20. Unexpected filesystem failures do not crash the app, falsely update the GUI, or silently lose data.
21. Existing JPEG sorter source code and behavior are preserved for later integration.

20. Required Tests

At minimum, add automated tests for:

- Valid and invalid project names, including Windows reserved names and 25-character limits.
- Case-insensitive duplicate detection.
- Collisions with unregistered filesystem entries.
- Project creation and rollback behavior.
- Rename success, collision failure, and capitalization-only rename behavior.
- Safe target validation for recursive deletion.
- Exact typed deletion-confirmation matching.
- Missing-folder detection and stale-entry removal.
- Recursive case-insensitive JPEG counting.
- Exclusion or safe handling of directory links/reparse points.
- Search, sort, and pagination calculations.
- Statistics clamping/display when counts are zero or inconsistent.
- Registry persistence and recovery from malformed data.
- Library-move collision blocking.
- Successful same-volume library move and registry-path updates.

- Simulated move/copy failures that verify no false success state or unsafe cleanup.

Also perform manual GUI verification at minimum window size, multiple larger sizes, common Windows display scaling, empty state, no-results state, missing-project state, long folder paths, and long-but-valid project names.

21. Explicit Future Integration Contract

Future project modules must:

- Store all user-visible files beneath the registered project folder.
- Notify the home-page statistics service after changing image or labeling data.
- Update last edited only for meaningful project changes.
- Count an image as labeled only when it has at least one bounding box or has been explicitly tagged `Empty`.
- Avoid depending on private metadata inside a project folder.

The final project folder structure and file-based method for recalculating labeled-image counts remain intentionally undefined. Until those requirements are written, the manual Refresh action updates total JPEG count and filesystem-derived last-edited time only; it does not infer labeling completion.